Lovable.dev Clone

Project Architecture, File Structure & Flow Guide

A complete blueprint for building an AI app-builder:
3-panel UI • Streaming LLM • Sandbox preview • Persistence

1. What You're Building

A Lovable.dev clone is a web app that lets users describe an app in natural language and watch it get built live. The user types a prompt; an LLM generates/edits code; the code runs in an isolated sandbox; the result is shown in an iframe preview — all in one screen.

Core Capabilities

Capability	What it does
Chat UI	User describes app in plain English; sees AI responses streaming.
AI Loop	LLM (Claude/GPT) reads request, plans, calls tools to write/edit files.
Sandbox	Each project runs in an isolated VM/container (E2B, Daytona, Modal).
Live Preview	Iframe shows the running app; updates as files change.
File Tree + Editor	Browse and edit generated code (Monaco editor).
Persistence	Projects, messages, files stored in Postgres.
Auth	Users sign in, own their projects.

Recommended Stack

Layer	Choice	Why
Framework	TanStack Start (or Next.js)	SSR + server functions in one place
Styling	Tailwind CSS + shadcn/ui	Fast, consistent UI primitives
LLM	Claude Sonnet / GPT-4o	Strong tool-calling, code quality
Sandbox	E2B / Daytona / WebContainers	Isolated runtime per project
Editor	Monaco	VS Code editor in the browser
Panels	react-resizable-panels	Draggable 3-panel layout
DB + Auth	Lovable Cloud (Supabase)	Postgres + Auth + Storage out of the box
Streaming	Server-Sent Events / streamText	Token-by-token AI responses

2. Project Folder Structure

This is the recommended layout (TanStack Start template). Each file has a clear single responsibility.

```
lovable-clone/
■■■ src/
■   ■■■ routes/                          # File-based routing
■   ■   ■■■ __root.tsx                   # App shell (html, head, providers)
■   ■   ■■■ index.tsx                    # Landing page ("/")
■   ■   ■■■ login.tsx                    # Auth page
■   ■   ■■■ _authenticated.tsx           # Layout: protects child routes
■   ■   ■■■ _authenticated/
■   ■   ■   ■■■ dashboard.tsx            # List of user's projects
■   ■   ■   ■■■ projects.$id.tsx         # The 3-panel builder UI
■   ■   ■■■ api/
■   ■       ■■■ public/
■   ■           ■■■ webhook.ts           # External webhooks (Stripe, etc.)
■   ■
■   ■■■ components/
■   ■   ■■■ ui/                          # shadcn/ui primitives (Button, Card...)
■   ■   ■■■ builder/
■   ■   ■   ■■■ ChatPanel.tsx            # Left: chat messages + input
■   ■   ■   ■■■ MessageBubble.tsx        # One chat message
■   ■   ■   ■■■ PromptInput.tsx          # Textarea + send button
■   ■   ■   ■■■ FileTree.tsx             # Middle: sandbox file explorer
■   ■   ■   ■■■ CodeEditor.tsx           # Monaco wrapper
■   ■   ■   ■■■ PreviewPane.tsx          # Right: iframe of running app
■   ■   ■   ■■■ PreviewToolbar.tsx       # Refresh / open / device toggle
■   ■   ■   ■■■ BuilderLayout.tsx        # Resizable 3-panel container
■   ■   ■■■ ProjectCard.tsx              # Dashboard tile
■   ■   ■■■ Header.tsx                   # Top nav
■   ■
■   ■■■ lib/
■   ■   ■■■ ai/
■   ■   ■   ■■■ system-prompt.ts         # The big system prompt for the LLM
■   ■   ■   ■■■ tools.ts                 # Tool defs: write_file, run_cmd...
■   ■   ■   ■■■ stream.functions.ts      # Server fn: streams AI response
■   ■   ■■■ sandbox/
■   ■   ■   ■■■ client.server.ts         # E2B/Daytona client (server only)
■   ■   ■   ■■■ manager.server.ts        # Create / wake / kill sandboxes
■   ■   ■   ■■■ files.functions.ts       # Read/write files in a sandbox
■   ■   ■■■ projects/
■   ■   ■   ■■■ projects.functions.ts    # CRUD: create/list/load projects
■   ■   ■   ■■■ messages.functions.ts    # Save chat history
■   ■   ■■■ utils.ts                     # cn(), small helpers
■   ■
■   ■■■ integrations/
■   ■   ■■■ supabase/
■   ■       ■■■ client.ts                # Browser Supabase client
■   ■       ■■■ client.server.ts         # Admin client (service role)
■   ■       ■■■ auth-middleware.ts       # requireSupabaseAuth
■   ■
■   ■■■ hooks/
■   ■   ■■■ use-chat.ts                  # Send msg + consume stream
■   ■   ■■■ use-sandbox.ts               # Track sandbox status + preview URL
■   ■   ■■■ use-project.ts               # Load project + subscribe to updates
■   ■
■   ■■■ styles.css                       # Tailwind + design tokens
■   ■■■ router.tsx                       # Router factory
■   ■■■ start.ts                         # Server middleware registration
```

```
■   ■■■ server.ts                          # SSR entry
■
■■■ supabase/migrations/                   # DB schema (projects, messages, files)
■■■ package.json
■■■ vite.config.ts
■■■ tsconfig.json
```

3. What Each File Does

Routes (Pages)

File	Responsibility
`__root.tsx`	Root layout. Sets up HTML shell, QueryClient, Toaster, auth provider. Renders <Outlet/>.
`index.tsx`	Public landing page with hero, examples, and 'Start building' CTA.
`login.tsx`	Email + OAuth sign-in form using Supabase Auth.
`_authenticated.tsx`	Pathless layout. beforeLoad redirects to /login if no session.
`dashboard.tsx`	Grid of the user's projects. 'New project' button creates a row + sandbox.
`projects.$id.tsx`	The MAIN builder screen. Loads project, mounts ChatPanel + FileTree + Preview.
`api/public/webhook.ts`	Public HTTP endpoints for external services. Verify signatures here.

Builder Components

File	Responsibility
`BuilderLayout.tsx`	Wraps the three panels in react-resizable-panels. Handles collapse/expand.
`ChatPanel.tsx`	Renders message history, calls use-chat hook, shows streaming tokens.
`MessageBubble.tsx`	One message: role (user/assistant), markdown body, tool-call cards.
`PromptInput.tsx`	Auto-growing textarea. Cmd+Enter to send. Disabled while AI is working.
`FileTree.tsx`	Lists files in the sandbox. Click → opens in CodeEditor.
`CodeEditor.tsx`	Monaco editor. Read-only by default; user can toggle edit mode.
`PreviewPane.tsx`	<iframe src={sandbox.previewUrl}>. Reloads when files change.
`PreviewToolbar.tsx`	Refresh, open-in-new-tab, mobile/desktop viewport toggle.

AI Logic (lib/ai)

File	Responsibility
`system-prompt.ts`	Long string telling the LLM: who it is, allowed tools, code style, output rules.
`tools.ts`	Tool schemas (Zod) the LLM can call: write_file, read_file, run_command, list_files.
`stream.functions.ts`	Server function. Receives a prompt + history, calls the LLM with tools, streams text + tool events back

Sandbox Logic (lib/sandbox)

File	Responsibility
`client.server.ts`	Initializes E2B/Daytona SDK with API key from env. Server-only.
`manager.server.ts`	createSandbox(projectId), getOrWake(projectId), killSandbox(). Stores sandboxId on the project row.

`files.functions.ts`	writeFile / readFile / listFiles inside a sandbox. Called by AI tools.

Project + Persistence

File	Responsibility
`projects.functions.ts`	createProject, listProjects, getProject. All use requireSupabaseAuth.
`messages.functions.ts`	appendMessage, listMessages. Replays history on page reload.

Integrations

File	Responsibility
`supabase/client.ts`	Browser client. Used for auth state listener and realtime.
`supabase/client.server.ts`	Service-role client. Used in webhooks and admin server work.
`supabase/auth-middleware.ts`	Server-function middleware that injects an auth'd supabase client.

Hooks

File	Responsibility
`use-chat.ts`	send(prompt) → calls stream.functions.ts → appends tokens to state.
`use-sandbox.ts`	Polls or subscribes to sandbox status. Exposes previewUrl + 'restart'.
`use-project.ts`	Loads project metadata; subscribes to file/message changes via realtime.

4. Database Schema

Stored in Lovable Cloud (Postgres). All tables have RLS — users can only see their own data.

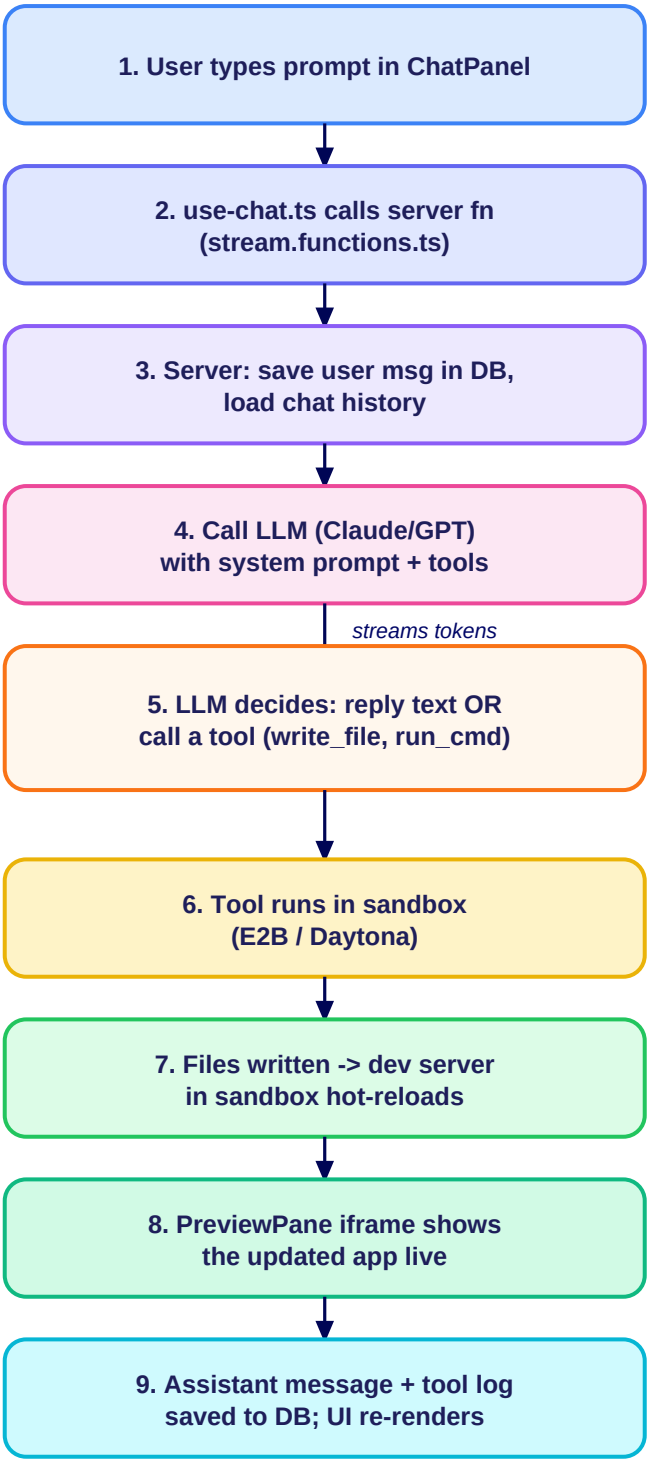
```
-- Each app the user builds
CREATE TABLE projects (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid REFERENCES auth.users(id) NOT NULL,
  name        text NOT NULL,
  sandbox_id  text,                -- E2B/Daytona sandbox handle
  preview_url text,                -- live iframe URL
  created_at  timestamptz DEFAULT now()
);

-- Chat history (one row per message)
CREATE TABLE messages (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id uuid REFERENCES projects(id) ON DELETE CASCADE,
  role       text CHECK (role IN ('user','assistant','tool')),
  content    jsonb NOT NULL,       -- text + tool calls + results
  created_at timestamptz DEFAULT now()
);

-- Optional: snapshot of files for fast reload without the sandbox
CREATE TABLE project_files (
  project_id uuid REFERENCES projects(id) ON DELETE CASCADE,
  path       text NOT NULL,
  content    text NOT NULL,
  updated_at timestamptz DEFAULT now(),
  PRIMARY KEY (project_id, path)
);
```

5. How It All Works (Flowchart)

End-to-end: from user typing a prompt to seeing the live preview update.


1. User types prompt in ChatPanel
2. use-chat.ts calls server fn (stream.functions.ts)
3. Server: save user msg in DB, load chat history
4. Call LLM (Claude/GPT) with system prompt + tools
streams tokens
5. LLM decides: reply text OR call a tool (write_file, run_cmd)
6. Tool runs in sandbox (E2B / Daytona)
7. Files written -> dev server in sandbox hot-reloads
8. PreviewPane iframe shows the updated app live
9. Assistant message + tool log saved to DB; UI re-renders

6. Recommended Build Order

Step 1 — Shell & Auth

Set up TanStack Start, Tailwind, shadcn/ui. Enable Lovable Cloud. Build /login and _authenticated layout. Create the projects table.

Step 2 — Dashboard

Build dashboard.tsx with 'New project' button. Create project row in DB; redirect to /projects/:id.

Step 3 — Builder UI (Mock)

Build projects.$id.tsx with BuilderLayout (3 resizable panels). ChatPanel + FileTree + Preview with hardcoded mock data.

Step 4 — Sandbox

Add E2B/Daytona. On project create, spin up sandbox; store sandbox_id and preview_url. PreviewPane shows the live iframe.

Step 5 — AI Loop

Write system-prompt.ts, tools.ts, stream.functions.ts. Hook ChatPanel to use-chat. Stream tokens; execute tools inside the sandbox.

Step 6 — Persistence

Save every message + tool result. On reload, replay history. Optional: snapshot files in project_files.

Step 7 — Polish

Loading states, error recovery, sandbox sleep/wake, file editor, deploy button, sharing.

Next step

Want me to scaffold Step 1 + Step 3 (auth shell + the mocked 3-panel builder UI) in this project right now? Just say the word.